

```
[localhost udp]$ gcc lfy_udp_client.c -o myclient
[localhost udp]$ ./myclient 127.0.0.1 11710
Client Starting service
Enter Data For the server
Testing
Data Sent To Server
Data Received from server 127.0.0.1:
Testing
Enter Data For the server
```

Though *client* and *server* generally mean processes running on different machines, in my example, both client and server are running on the same Fedora 15 machine. The server process typically listens on a particular 'port number'; in this case, I have randomly chosen 11710. The client should know the IP (Internet Protocol) address of the machine on which the server is running, as well as the port number. (Here the address is 127.0.0.1, the loop-back address, since I am using the same machine to run the client and the server.)

The client requests some service from the server, which serves the client and typically waits for another client request. In this example, the 'service' is simply returning the string it received from the client and this is displayed by the client. After running the server, you can use the *netstat* command to see the socket opened by the server:

```
[localhost udp]$ netstat -a | grep "11710"
udp      0      0 *:11710      :*
```

The output shows that one UDP socket is available, bound to port 11710. Please remember that the *netstat* command is very useful when you work with sockets.

Now let us go over the simple algorithms for our UDP client and server.

The algorithm for a simple UDP client

```
Get the socket descriptor (using the socket function).
Update a socket address structure with the server's IP address
and port.
Get data for sending in the datagram.
Send datagram to server, using the address of the server.
Receive datagram from server. (Here, the process waits—that is,
gets blocked, until the recvfrom function returns).
Close the socket descriptor.
```

The algorithm for a simple UDP server

```
Get the socket descriptor (using the socket function).
Update a socket address structure with the local address and port
number.
Associate (bind) the above address with the socket.
Repeat forever {
```

```
    Receive datagram from a client (Here, the process waits—that
    is, gets blocked, until the recvfrom function returns).
    Process received data.
    Send datagram to the client, from whom datagram was
    received.
}
```

The source code

Now please refer to the downloaded source code. The socket function returns a socket descriptor (IPv4, *SOCK_DGRAM*(*UDP*)), referred to by the variable *sd*. An address structure *struct sockaddr_in* (whose declaration is in the file */usr/include/netinet/in.h* in Linux) is needed. It's declaration is available in the source code by inclusion of *<netinet/in.h>*.

In the client code, you need to populate this address structure (the *serveraddress* variable) with the address family, the server's IP address, and the port number. Note that this needs to be populated in the big-endian order, so use functions/macros like *htons* and *inet_addr*—*htons*(*atoi(argv[2])*), which transform the string the user passes as the command-line argument into a big-endian short integer. Similarly, *inet_addr(argv[1])* transforms the IP address command-line argument into a 32-bit IPv4 address in the big-endian order.

```
serveraddress.sin_family = AF_INET;
serveraddress.sin_port = htons(atoi(argv[2])); //PORT NO
serveraddress.sin_addr.s_addr = inet_addr(argv[1]); //ADDRESS
```

I used the functions *recvfrom* and *sendto* to receive and send UDP datagrams respectively. The *sendto* function prototype is as follows:

```
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);
```

The first argument is the socket descriptor, the second is the array from where to send data, and the third is the number of bytes to send. We have the fourth argument (*flags*) as 0. This supports a few options, which you can learn from the manual page. The fifth argument is the address structure, which needs to be populated (in the client, it is populated with the server's IP address and port number). The sixth argument is the size of this address structure.

Let us now understand the *recvfrom* function. Its prototype is as shown below:

```
ssize_t recvfrom(int s, void *buf, size_t len, int flags, struct
sock-
```

```
addr *from, socklen_t *fromlen);
```

The first three arguments are, respectively, the socket descriptor, the array where the received data is to be stored, and the length of that array. The fourth is the *flags* argument,